PYTHON PROGRAMMING LAB - ASSIGNMENT 2

Design, Implement and Analyse Python Programming Concepts

Assignment report (Questions 1-7)

Name	P. VAMSI
Register number	1925260294
Course	Python Programming
Python version	3.12 (standard library)
Assignment	CO2 Coverage - Python Programming Lab
GitHub repository	https://github.com/vamsinani889-svg/PYTHON-PROGRAMMING-ASSIGNMENT2

Q1 — Decisions with Many Boundaries (6 Marks)

Problem: Electricity subsidy eligibility depends on household consumption and whether the household has a ration card or a member above 80 years of age.

1. Rule interpretation

Condition	Decision
Consumption ≤ 100 units	**Eligible for subsidy.**
101 to 200 units	**Partial subsidy only when the household has a ration card.**
Consumption > 200 units	**Not eligible, except when a household member is above 80 years.**

The question does not specify the amount of the partial subsidy or the special subsidy for the age exception. Therefore, the program reports eligibility categories rather than inventing monetary amounts.

2. Predictions for eight households

Household	Units	Ration card?	Member >80?	Predicted result
H1	**99**	**No**	**No**	**Full subsidy**
H2	**100**	**Yes**	**No**	**Full subsidy**
H3	**101**	**No**	**No**	**No subsidy**
H4	**150**	**Yes**	**No**	**Partial subsidy**
H5	**200**	**No**	**No**	**No subsidy**
H6	**200**	**Yes**	**No**	**Partial subsidy**
H7	**201**	**No**	**Yes**	**Age-exception subsidy**
H8	**250**	**Yes**	**No**	**No subsidy**

3. Implementation using separate if statements

```
def subsidy_separate(units, ration_card, age_over_80):
    results = []

    if units <= 100:
        results.append("Full subsidy")

    if 101 <= units <= 200 and ration_card:
        results.append("Partial subsidy")

    if units > 200 and age_over_80:
        results.append("Age-exception subsidy")

    if not results:
        results.append("No subsidy")

    return results

households = [
    (99, False, False), (100, True, False),
    (101, False, False), (150, True, False),
    (200, False, False), (200, True, False),
    (201, False, True), (250, True, False)
]

for number, data in enumerate(households, start=1):
    print(f"H{number}: {subsidy_separate(*data)}")
```

4. Implementation using if-elif-else

```
def subsidy_chain(units, ration_card, age_over_80):

    if units <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200:
        if ration_card:
            return "Partial subsidy"
        else:
            return "No subsidy"
    elif units > 200 and age_over_80:
        return "Age-exception subsidy"
    else:
        return "No subsidy"

for number, data in enumerate(households, start=1):
    print(f"H{number}: {subsidy_chain(*data)}")
```

Expected decisions: H1 Full; H2 Full; H3 None; H4 Partial; H5 None; H6 Partial; H7 Age exception; H8 None.

5. Why separate if statements can produce multiple results

Separate if statements are independent. After one condition is true, Python continues checking the next if statement. If more than one condition is true, more than one action may run. An if-elif-else chain checks one branch and stops after the first true condition, so it produces exactly one decision.

6. Design decision and alternative concepts

A single if-elif-else chain with a nested ration-card check is easiest to read because the consumption bands are mutually exclusive. A match-case alternative can classify fixed categories, but numeric range matching needs guards. A lookup-list alternative can store upper limits and decisions, then scan until the first matching limit.

```
def subsidy_match(units, ration_card, age_over_80):
    match units:
        case n if n <= 100:
            return "Full subsidy"
        case n if 101 <= n <= 200 and ration_card:
            return "Partial subsidy"
        case n if 101 <= n <= 200:
            return "No subsidy"
        case n if n > 200 and age_over_80:
            return "Age-exception subsidy"
        case _:
            return "No subsidy"

def subsidy_lookup(units, ration_card, age_over_80):
    rules = [
        (100, "Full subsidy"),
        (200, "Partial subsidy"),
        (float("inf"), "No subsidy")
    ]
    if units > 200 and age_over_80:
        return "Age-exception subsidy"
    for upper_limit, decision in rules:
        if units <= upper_limit:
            if upper_limit == 200 and not ration_card:
                return "No subsidy"
            return decision
```

Complexity: The if-elif chain checks a fixed number of conditions, so time is O(1) and auxiliary space is O(1). The lookup method checks at most k rules, so time is O(k) and auxiliary space is O(1) when the rule list is treated as fixed.

Conclusion: Use if-elif-else for this small, mutually exclusive rule set. It avoids accidental multiple decisions and makes boundary handling explicit.

Q2 — Loops that Count Attempts (4 Marks)

Problem: A bank kiosk allows three OTP attempts. After three incorrect attempts, the account is locked for 30 minutes. A blank entry must not count as an attempt.

1. while-loop implementation

```
def verify_otp_while(correct_otp):
    attempts = 0

    while attempts < 3:
        entered = input("Enter OTP: ").strip()

        if entered == "":
            print("Blank entry: attempt not counted.")
            continue

        attempts += 1

        if entered == correct_otp:
            print("OTP verified.")
            break
        else:
            print("Incorrect OTP.")

    else:
        print("Account locked for 30 minutes.")

verify_otp_while("4821")
```

2. for-loop implementation using range(3)

```
def verify_otp_for(correct_otp):
    for attempt in range(3):
        entered = input("Enter OTP: ").strip()

        if entered == "":
            print("Blank entry: attempt not counted.")
            # This loop iteration is consumed by range(3).
            # To ensure blank entries do not consume attempts,
            # use a separate counter or a while loop instead.
            continue

        if entered == correct_otp:
            print("OTP verified.")
            break
        print("Incorrect OTP.")
    else:
        print("Account locked for 30 minutes.")
```

The direct range(3) version above illustrates the limitation: continue moves to the next for-loop iteration, so a blank response consumes one of the three iterations. To satisfy the exact rule, use a counted while loop, or use a for loop over an open-ended iterator with a separate successful-attempt counter.

Correct for-loop alternative with a separate counter

```
from itertools import count

def verify_otp_for_correct(correct_otp):
    attempts = 0

    for _ in count():
        if attempts == 3:
            print("Account locked for 30 minutes.")
            break

        entered = input("Enter OTP: ").strip()

        if entered == "":
            print("Blank entry: attempt not counted.")
            continue

        attempts += 1

        if entered == correct_otp:
            print("OTP verified.")
            break

        print("Incorrect OTP.")
```

3. break, continue, pass and loop else

• break: Ends the loop immediately when the correct OTP is entered.

• continue: Skips the remaining statements for a blank input and starts the next iteration without increasing the attempts counter.

• pass: Does nothing. It can be used as a temporary placeholder for an optional logging or audit branch, but it is not needed for the core verification logic.

• loop else: Runs only when the loop finishes normally, without break. It is suitable for displaying the lockout message after all allowed attempts fail.

4. When is while more natural than for?

A while loop is more natural because the number of input prompts is not fixed: blank entries do not consume attempts, and the loop continues until three non-blank entries are received or the correct OTP is entered.

5. What can go wrong if the counter is updated in the wrong place?

If attempts is increased before checking for a blank entry, blank inputs will incorrectly reduce the available attempts. If the counter is never increased for incorrect non-blank entries, the loop may continue indefinitely.

6. Changed condition: two extra attempts after a correct security answer

When the customer answers the security question correctly, the maximum number of non-blank OTP attempts becomes five (three normal attempts plus two extra attempts). Blank entries still do not count.

```
def verify_with_security(correct_otp, security_answer_correct):
    max_attempts = 5 if security_answer_correct else 3
    attempts = 0

    while attempts < max_attempts:
        entered = input("Enter OTP: ").strip()

        if not entered:
            print("Blank entry ignored.")
            continue

        attempts += 1
        if entered == correct_otp:
            print("OTP verified.")
            return True

        print("Incorrect OTP.")

    print("Account locked for 30 minutes.")
    return False
```

7. Recursive alternative and complexity

A recursive design would call a function again for the next attempt. It is possible, but it uses one stack frame per attempt and makes blank-input handling less straightforward.

```
def verify_recursive(correct_otp, attempts_left):
    if attempts_left == 0:
        print("Account locked for 30 minutes.")
        return False

    entered = input("Enter OTP: ").strip()
    if not entered:
        print("Blank entry ignored.")
        return verify_recursive(correct_otp, attempts_left)

    if entered == correct_otp:
        print("OTP verified.")
        return True

    print("Incorrect OTP.")
    return verify_recursive(correct_otp, attempts_left - 1)
```

Complexity: For a attempts, loop time is O(a) and auxiliary space is O(1). Recursive time is O(a), but call-stack space is O(a). Here a is at most 3 or 5, so both are small; the loop is simpler and avoids recursion overhead.

Q3 — Engineer a Vehicle Registration Number Validator (6

Marks) Problem: Toll-plaza records may contain the same registration number in different formats. Normalize, validate and print valid numbers in the form TN 09 AB 1234. Regular expressions are not permitted.

1. Format rules

• 2 letters for the state code.

• 2 digits for the district code.

• 1 or 2 letters for the series.

• 4 digits for the registration number.

• After removing spaces and hyphens and converting to uppercase, the valid length is 9 or 10 characters.

2. Completed code using permitted string operations

```
def normalise(text):
    # Remove spaces and hyphens, then convert to uppercase.
    return text.replace(" ", "").replace("-", "").strip().upper()

def is_valid(reg):
    # Check total length first.
    if len(reg) not in (9, 10):
        return False

    state = reg[0:2]
    district = reg[2:4]
    series = reg[4:-4]
    number = reg[-4:]

    return (
        len(state) == 2 and state.isalpha()
        and len(district) == 2 and district.isdigit()
        and len(series) in (1, 2) and series.isalpha()
        and len(number) == 4 and number.isdigit()
    )

def format_number(reg):
    if not is_valid(reg):
        return "INVALID"
    return f"{reg[0:2]} {reg[2:4]} {reg[4:-4]} {reg[-4:]}"

samples = [
    "tn 09 ab 1234",
    "KA-01-M-0007",
    "TN09AB1234",
    "TN 22 1234 AB",
    "T9-09-AB-1234",
    "KA 1 M 0007"
]

for raw in samples:
    reg = normalise(raw)
    print(f"{raw:18} -> {reg:10} "
          f"{'VALID' if is_valid(reg) else 'INVALID'}")

print(format_number(normalise("tn 09 ab 1234")))
```

3. Expected output

```
tn 09 ab 1234         -> TN09AB1234 VALID
KA-01-M-0007          -> KA01M0007 VALID
TN09AB1234            -> TN09AB1234 VALID
TN 22 1234 AB         -> TN221234AB INVALID
T9-09-AB-1234         -> T909AB1234 INVALID
KA 1 M 0007           -> KA1M0007   INVALID
TN 09 AB 1234
```

4. Explanation of string concepts

• Immutability: Python strings cannot be changed in place. String methods return new strings.

• upper(): text.upper() creates an uppercase string; it does not modify the original variable.

• Slicing: reg[0:2] selects the first two characters. reg[-4:] selects the final four characters. reg[4:-4] selects the series characters.

For a normalized valid value, s[0:2] returns the state, s[2:4] returns the district, s[4:-4] returns the series, and s[-4:] returns the registration digits.

5. Alternative using a regular expression

```
import re

def is_valid_regex(reg):
    return re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg) is not None
```

The regular expression is compact, but it is not permitted for the main implementation. The slicing approach makes each field visible and is easy for a beginner to debug.

Complexity: Normalization and validation inspect a string of length L, so time is O(L). The normalized string and slices require O(L) auxiliary space in the general case. Both approaches are linear; the string-method approach is more explicit for this assignment.

Q4 — Building Long Text: + or join()? (6 Marks)

Problem: A mobile operator builds monthly SMS statements by combining many lines. Compare repeated +=, collecting parts and using join(), and an f-string-based line generator.

1. Three implementations

```
import time

items = [f"Item {i}: Rs.{i % 50}" for i in range(200_000)]

def build_plus(parts):
    text = ""
    for part in parts:
        text += part + "\n"
    return text

def build_join(parts):
    return "\n".join(parts) + "\n"

def build_fstring(parts):
    return "".join(f"{part}\n" for part in parts)

def measure(function, parts, repeats=3):
    times = []
    result = None
    for _ in range(repeats):
        start = time.perf_counter()
        result = function(parts)
        times.append(time.perf_counter() - start)
    return result, sum(times) / len(times)

for n in (1_000, 10_000, 200_000):
    sample = items[:n]
    a, ta = measure(build_plus, sample)
    b, tb = measure(build_join, sample)
    c, tc = measure(build_fstring, sample)

     print(f"Lines: {n:,}")
     print(f"+=       {ta:.6f} seconds")
     print(f"join     {tb:.6f} seconds")
     print(f"f-string {tc:.6f} seconds")
     print("Same output:", a == b == c)
```

2. Prediction

The join() and f-string generator approaches are expected to scale well because they assemble the final string from parts. Repeated += may repeatedly create and copy strings. CPython can optimize some local += patterns, so measured results may be closer than the simple theoretical model suggests.

3. Why join() is recommended

Strings are immutable. A concatenation creates a new string value. Repeatedly extending a growing string can require copying the existing characters again and again. With join(), Python knows the collection of parts and can construct the result efficiently in one assembly operation.

4. Complexity comparison

Method	Typical time model	Extra space	Comment
Repeated += in a loop	Can approach O(n²) total copied characters	O(total output)	May be optimized in some CPython cases; performance is implementation-dependent.
List + join()	O(n + total characters)	O(total output + list)	Recommended for collecting many pieces.
Generator + join()	O(n + total characters)	O(total output), plus small generator state	Avoids storing a separate list of all formatted lines.
Write each line to a file	O(total characters)	O(buffer size)	Useful when output is too large to keep fully in memory.

5. Alternative: stream lines directly to a file

```
def write_statement(parts, filename):
    with open(filename, "w", encoding="utf-8") as file:
        for part in parts:
            file.write(f"{part}\n")

write_statement(items, "monthly_statement.txt")
```

File streaming avoids holding the complete statement string in memory. It is useful for very large reports, but it creates a file and is less convenient when the complete string must be returned to another function.

Evidence to record: Run the benchmark three times for 1,000, 10,000 and 200,000 lines. Record the average values printed by your computer. Do not copy another computer's timings as your own.

Q5 — Functions, Parameters and Scope (7 Marks)

Problem: A canteen order function uses add_item(item, order=[]). The default list is created once when the function is defined, so later calls can share the same list.

1. Demonstrate the bug

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

print(add_item_buggy("Tea"))
print(add_item_buggy("Vada"))
print(add_item_buggy.__defaults__)
```

Expected observation: the second call contains both Tea and Vada because the same default list is reused. The default list is stored in the function's __defaults__ attribute.

2. Correct implementation

```
def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

print(add_item("Tea"))
print(add_item("Vada"))
print(add_item("Coffee", ["Sandwich"]))
```

Expected output:

```
['Tea']
['Vada']
['Sandwich', 'Coffee']
```

3. Three customers tested in sequence

```
customer_1 = add_item("Tea")
customer_2 = add_item("Vada")
customer_3 = add_item("Coffee")

print("Customer 1:", customer_1)
print("Customer 2:", customer_2)
print("Customer 3:", customer_3)
```

Each call receives a new list when order is None, so one customer's order does not leak into another customer's order.

4. Positional, keyword and default parameters

• Positional: values are assigned according to their order, e.g. add_item('Tea', order_list).

• Keyword: values are named, e.g. add_item(item='Tea', order=order_list).

• Default: a value is used when the caller omits that argument, e.g. order=None.

5. Returning more than one value

```
def order_summary(items, price_per_item):
    total = len(items) * price_per_item
    gst = total * 0.05
    return total, gst

total, gst = order_summary(["Tea", "Vada"], 20)
print("Total:", total)
print("GST:", gst)
```

Python returns the two values as a tuple. The caller can unpack that tuple into separate variables.

6. Local, global and nonlocal scope

```
tax_rate = 0.05   # global variable

def make_counter():
    count = 0       # enclosing (nonlocal) variable

    def increment():
        nonlocal count
        count += 1
        return count

    return increment

def calculate_total(amount):
    tax = amount * tax_rate # local variable tax; reads global tax_rate
    return amount + tax

counter = make_counter()
print(counter(), counter(), counter())
print(calculate_total(100))
```

• A local variable is created inside a function and is normally available only there.

• A global variable is defined at module level. Use global only when a function must reassign that module-level name.

• A nonlocal variable belongs to an enclosing function. nonlocal allows an inner function to reassign it.

• Changing global state is usually avoided because it makes functions harder to test, reuse and reason about. Passing values as parameters and returning results is usually clearer.

7. Alternative designs and complexity

The caller can always pass an explicit list, or a closure can create and retain a separate order list for one customer. The None-default pattern is the simplest general-purpose design.

Complexity: Appending one item to a Python list is O(1) amortized time. A new empty list requires O(1) space, while the order itself occupies O(m) space for m items. Each customer gets a separate list in the corrected design.

Q6 — Recursion and Its Limits (8 Marks)

Problem: A devotee can climb one or two steps at a time. Count the number of different ways to reach the top of n steps, for staircases up to 1,000 steps.

1. Recurrence and base cases

Let W(n) be the number of ways to reach step n. The final move must come from step n–1 or n–2, so W(n) = W(n–1) + W(n–2). Use W(0) = 1 and W(1) = 1. Thus W(2) = 2.

2. Plain recursive solution

```
calls = 0

def ways_naive(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n - 1) + ways_naive(n - 2)

calls = 0
print("ways:", ways_naive(5))
print("calls:", calls)
```

For n = 5, the answer is 8. The call tree repeats work: for example, ways_naive(3) is calculated more than once.

3. Memoised recursive solution

```
from functools import lru_cache

memo_calls = 0

@lru_cache(maxsize=None)
def ways_memo(n):
    global memo_calls
    memo_calls += 1

    if n <= 1:
        return 1

    return ways_memo(n - 1) + ways_memo(n - 2)

memo_calls = 0
print("ways:", ways_memo(5))
print("calls:", memo_calls)
```

Memoisation stores a result after its first calculation. A later request for the same n reuses the cached result instead of expanding the same recursive tree again.

4. Manual correctness check for n = 1 to 6

n	Calculation	Ways
1	1	1
2	1+1	2
3	2+1	3
4	3+2	5
5	5+3	8
6	8+5	13

5. Growth and call-count experiment

```
from functools import lru_cache

def naive_count(n):
    calls = 0
    def solve(k):
        nonlocal calls
        calls += 1
        if k <= 1:
            return 1
        return solve(k - 1) + solve(k - 2)
    answer = solve(n)
    return answer, calls

def memo_count(n):
    calls = 0
    cache = {}
    def solve(k):
        nonlocal calls
        calls += 1
        if k in cache:
            return cache[k]
        if k <= 1:
            result = 1
        else:
            result = solve(k - 1) + solve(k - 2)
        cache[k] = result
        return result
    answer = solve(n)
    return answer, calls

for n in (10, 20, 25, 30):
    answer, naive_calls = naive_count(n)
    memo_answer, memo_calls = memo_count(n)
    print(n, answer, naive_calls, memo_calls)
```

The exact counts from this code depend on the chosen counting convention (the code counts every function entry, including cache hits). The memoised answer is the same as the naive answer, while the number of calls is dramatically smaller.

6. Why memoised recursion can fail at n = 1,000

Memoisation reduces repeated calculations, but it does not remove recursive call depth. Python has a recursion limit to prevent excessive call-stack use. A call such as ways_memo(1000) may raise RecursionError because the dependency chain can be about 1,000 calls deep. Raising the limit is not a robust general solution.

7. Iterative solution using only the last two values

```
def ways_loop(n):
    if n < 0:
        raise ValueError("n must be non-negative")

    previous_two = 1   # W(0)
    previous_one = 1   # W(1)

    if n == 0:
        return previous_two

    for _ in range(2, n + 1):
        current = previous_one + previous_two
        previous_two = previous_one
        previous_one = current

    return previous_one

for n in range(1, 7):
    print(n, ways_loop(n))

print("n=1000:", ways_loop(1000))
```

8. Complexity comparison

Approach	Time complexity	Auxiliary space	Main limitation / benefit
Plain recursion	O(2^n)	O(n) stack	Repeated calculations; impractical for larger n.
Memoised recursion	O(n)	O(n) cache + O(n) stack	Fast, but still subject to recursion depth.

Iterative loop	O(n)	O(1)	Efficient and avoids

```
                                                                    recursion-limit
errors.
```

Deployment decision: Use the iterative loop for n up to 1,000 and beyond. It computes each value once, keeps only two previous values and does not depend on Python's recursion limit.

Q7 — Diagnose, Redesign and Validate (13 Marks)

Selected program: Q6, the temple-stair counting program. This program is a strong redesign candidate because the plain recursive version repeats the same calculations as n grows, and even memoised recursion retains a deep call stack.

A. Diagnose the bottleneck — 2 Marks

• Most repeated work in the original program occurs in the recursive calls ways_naive(n - 1) and ways_naive(n - 2).

• The call count grows exponentially with n because the same subproblems are solved repeatedly.

• Memoisation removes repeated subproblem calculations, but the recursive dependency chain still uses stack frames.

• The limitation is algorithmic, not merely a slow Python statement: the naive recurrence performs exponential work.

Evidence: Run the Q6 call-count program for n = 10, 20, 25 and 30. The naive call count rises rapidly, while the memoised version performs a number of new calculations proportional to n.

B. Define the design objective — 2 Marks

Redesign the stair-counting program so that it returns exactly the same answer for every non-negative n, runs in O(n) time, uses O(1) auxiliary space, and avoids recursive calls so that n = 1,000 can be handled without changing Python's recursion limit.

C. Design the revised solution — 4 Marks

What changes: Replace recursive function calls with a loop. Keep only W(n–2) and W(n–1), calculate the next value, then shift the two stored values forward.

Property exploited: The recurrence depends only on the previous two results. Older values are not required after the next result has been calculated.

Extra memory: No cache or recursive call stack is needed. Only a fixed number of variables are stored.

Trade-off: The loop is less directly similar to the mathematical recursive definition, but it is more scalable and avoids recursion depth errors.

Pseudocode

```
START
    READ n
    IF n < 0
        DISPLAY "n must be non-negative"
        STOP
    END IF

      previous_two <- 1
      previous_one <- 1

      IF n = 0
          DISPLAY previous_two
          STOP
      END IF

      FOR step FROM 2 TO n
          current <- previous_two + previous_one
          previous_two <- previous_one
          previous_one <- current
      END FOR

      DISPLAY previous_one
END
```

D. Predict before implementing — 1 Mark

Input n	Predicted number of ways
0	1

Input n	Predicted number of ways
1	1
2	2

3	**3**
4	**5**
5	**8**
6	**13**
1000	**A very large integer; calculated without recursion.**

The prediction is that the iterative version returns the same values as the recursive and memoised versions for small n, while avoiding recursion-limit failure for n = 1,000.

E. Implement and validate the redesign — 2 Marks

```
def ways_iterative(n):
    if n < 0:
        raise ValueError("n must be non-negative")

      a, b = 1, 1
      if n == 0:
          return a

      for _ in range(2, n + 1):
          a, b = b, a + b

      return b

# Compare all approaches for manageable inputs.
for n in range(0, 31):
    assert ways_iterative(n) == ways_memo(n)

print("Validation passed for n = 0 through 30.")
print("Iterative result for n = 1000 has", len(str(ways_iterative(1000))), "digits.")
```

The assertions compare the redesigned loop with the memoised reference implementation for n = 0 through 30. The final line demonstrates that n = 1,000 is computed and reports the number of digits in the result.

F. Defend the final design — 2 Marks

Measure	Plain recursion	Memoised recursion	Iterative redesign
Correct result	**Correct**	**Correct**	**Correct**
Time	**O(2^n)**	**O(n)**	**O(n)**
Auxiliary space	**O(n) stack**	**O(n) cache + stack**	**O(1)**
Repeated work	**High**	**Avoided by cache**	**Avoided by loop**
Recursion limit	**Can be reached**	**Can be reached**	**Not applicable**
Best use	**Small teaching example**	**When recursive structure is**	**Large n and production use**

```
                                                useful
```

Final decision: The iterative design is selected. It preserves the recurrence and exact answer while improving time complexity from exponential to linear compared with plain recursion. Compared with memoisation, it retains O(n) time but reduces auxiliary space from O(n) to O(1) and removes recursion-depth risk.

Overall Conclusion The seven tasks demonstrate how Python control structures and data structures affect correctness, readability and performance. Boundary-based decisions should use mutually exclusive branches; attempt counters must be updated only for valid attempts; string processing can be performed with slicing and built-in string methods; join() is suitable for assembling large text; mutable default arguments should be avoided; and recursion should be selected with care.

The strongest redesign in Q7 is the iterative stair-counting algorithm. It avoids repeated work, avoids the recursion limit and stores only the previous two results.

Final execution and submission checklist • Run every code block in Python 3.10 or later and check the output.

• For Q4, run the benchmark on your own computer and paste your actual average timings into the report.

• Capture screenshots showing code execution and output for each task.

• Save each task's code in its own Task01–Task07 folder.

• Create README.md and requirements.txt if any external libraries are used (these programs mainly use the Python standard library).

• Commit each task separately and include the GitHub repository link in the final assignment submission. Student note: Review the code, run it yourself, and be prepared to explain each condition, loop, function and complexity result during lab evaluation.